



EDUCACIÓN
SECRETARÍA DE EDUCACIÓN PÚBLICA



TECNOLÓGICO
NACIONAL DE MÉXICO



Instituto Tecnológico de San Juan del Río



Tópicos de Ciberseguridad

Reconstrucción del código fuente con Ghidra y comparación de ejecutables

P R E S E N T A N:

Daniel Olvera Romero - 22590344

Luis Fernando Loma Navarrete - 22590333

Jose Pedro Cruz Ramírez - 22590322

Ingeniería en Sistemas Computacionales

Periodo: Agosto - Diciembre 2026



Tabla de contenido

1. Objetivo.....	4
2. Herramientas utilizadas.....	4
3. Desarrollo de R003 - Reconstrucción del código fuente	4
3.1 Archivo de análisis.....	4
3.2 Creación del proyecto en Ghidra	5
3.3 Importación del ejecutable.....	7
3.4 Análisis automático y localización del main	9
3.5 Decompilación del pseudocódigo	11
3.6 Escritura del código fuente en C	13
4. Desarrollo de R004 - Compilación y comparación.....	15
4.1 Compilación del código reconstruido	15
4.2 Comprobación funcional.....	16
4.3 Comparación de tamaño y hash SHA-256.....	18
5. Resultados	18
6. Conclusiones	19



Índice de figuras

Propiedades del archivo desconocido.exe.....	5
Creación de un nuevo proyecto no compartido en Ghidra	6
Configuración del nuevo proyecto no compartido en Ghidra.....	6
Asignación del nombre y la carpeta de trabajo del proyecto.	7
Cuadro de diálogo de importación de desconocido.exe (File → Import File).....	7
Detección automática del formato Portable Executable (PE) de 32 bits, arquitectura x86.	8
Ventana Import Results Summary con el resumen del archivo importado.	9
Cuadro de diálogo de análisis automático (Analyze) de Ghidra.....	10
Opciones de análisis marcadas por defecto antes de ejecutar Analyze.	10
Symbol Tree, carpeta Functions, mostrando FUN_00401000 seleccionada.	11
Symbol Tree expandido: FUN_00401000 con sus parámetros y variables locales	11
Cuadro Edit Function Signature antes de la corrección	12
Cuadro Edit Function Signature después de la corrección (main, int argc, char **argv).....	13
Archivo sumnotransfer.c abierto en el editor de código (parte 1).	14
Archivo sumnotransfer.c abierto en el editor de código (parte 2).	15
Compilación de prueba.c con TCC para generar reconstruido.exe.	16
Ejecución de desconocido.exe con el archivo prueba.txt.	17
Ejecución de reconstruido.exe con el archivo prueba.txt.	17
Resultado de dir y certutil -hashfile para ambos ejecutables.....	18



1. Objetivo

Analizar el ejecutable desconocido.exe mediante Ghidra para reconstruir en lenguaje C la lógica observada en el Decompiler. Posteriormente, recompilar el código reconstruido con Tiny C Compiler (TCC) y comparar el nuevo ejecutable con el original mediante funcionamiento, tamaño y hash SHA-256.

La práctica se divide en dos reactivos: R003, enfocado en documentar el proceso de reconstrucción del código fuente; y R004, orientado a compilar el código reconstruido y comparar el ejecutable resultante con el binario original.

2. Herramientas utilizadas

- Windows y PowerShell / cmd para ejecutar comandos y validar archivos.
- Tiny C Compiler (TCC) 0.9.27) para compilar los programas en C.
- Ghidra 12.1.3 para el análisis estático y la descompilación del ejecutable.
- Java (OpenJDK) como entorno requerido para ejecutar Ghidra.
- Editor de código para escribir el archivo fuente en C reconstruido.

3. Desarrollo de R003 - Reconstrucción del código fuente

3.1 Archivo de análisis

El docente proporcionó el archivo desconocido.exe (2048 bytes, PE32 para Windows, Intel i386), el cual no cuenta con información de depuración ni con el código fuente original. El objetivo es recuperar su lógica mediante ingeniería inversa.

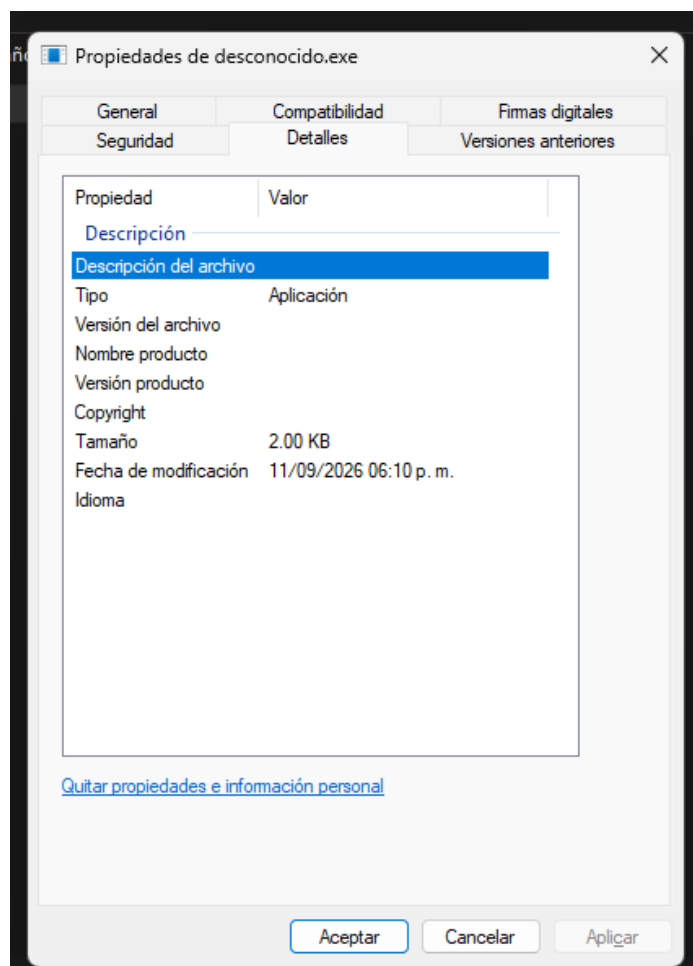


Figura 1. *Propiedades del archivo desconocido.exe*

3.2 Creación del proyecto en Ghidra

Se abrió Ghidra (ghidraRun.bat) y se creó un nuevo proyecto no compartido (File → New Project → Non-Shared Project), asignándole un nombre y una carpeta de trabajo.

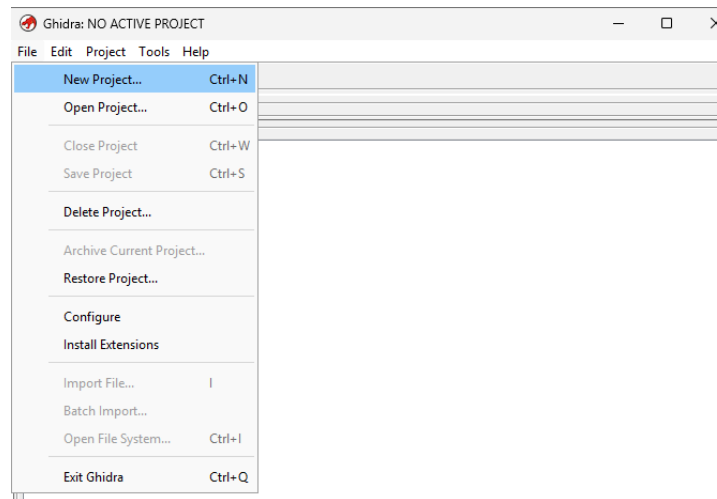


Figura 2. Creación de un nuevo proyecto no compartido en Ghidra

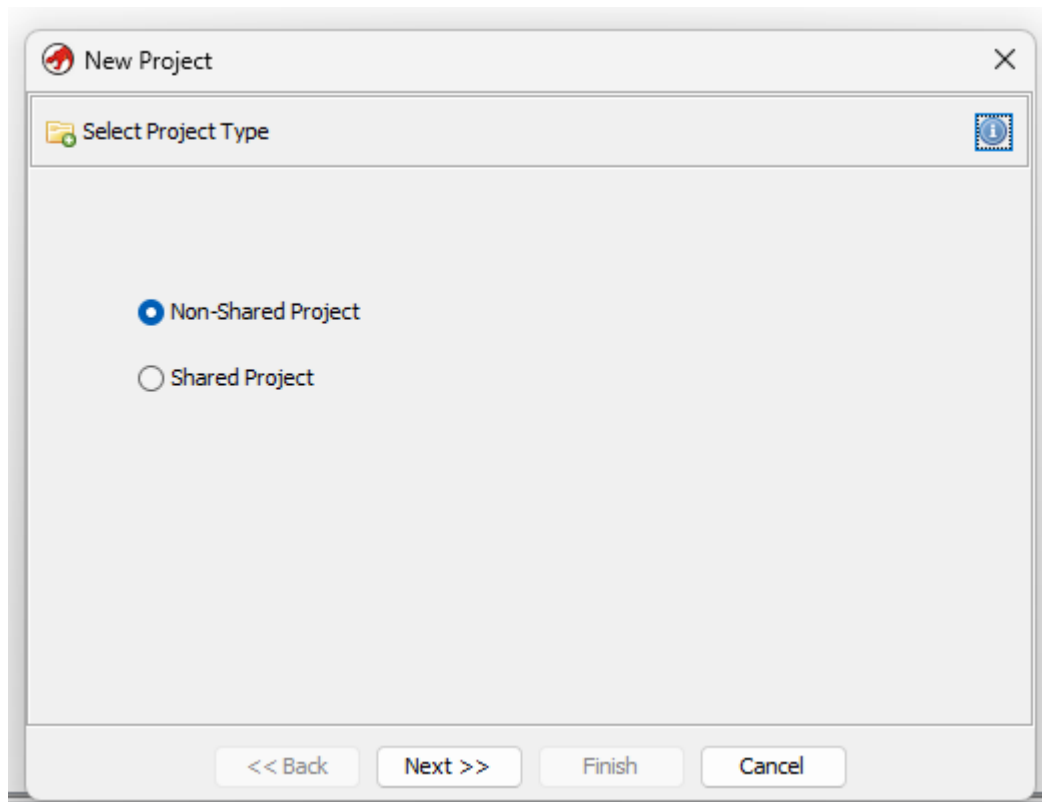


Figura 3. Configuración del nuevo proyecto no compartido en Ghidra.

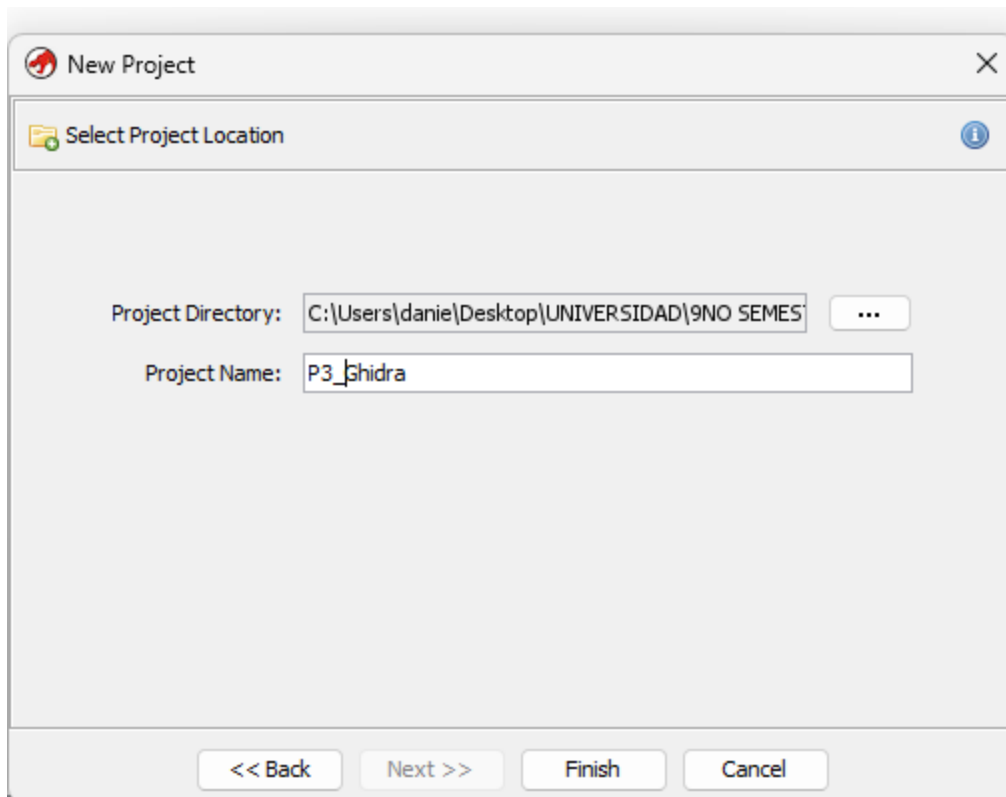


Figura 4. Asignación del nombre y la carpeta de trabajo del proyecto.

3.3 Importación del ejecutable

Con File → Import File se importó desconocido.exe. Ghidra detectó automáticamente el formato Portable Executable (PE) de 32 bits, arquitectura x86, little-endian, para Windows, por lo que se dejó la configuración por defecto y se aceptó con OK.

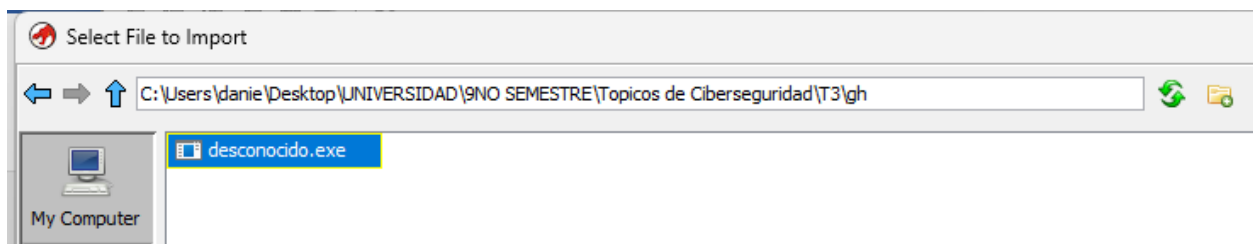


Figura 5. Cuadro de diálogo de importación de desconocido.exe (File → Import File).

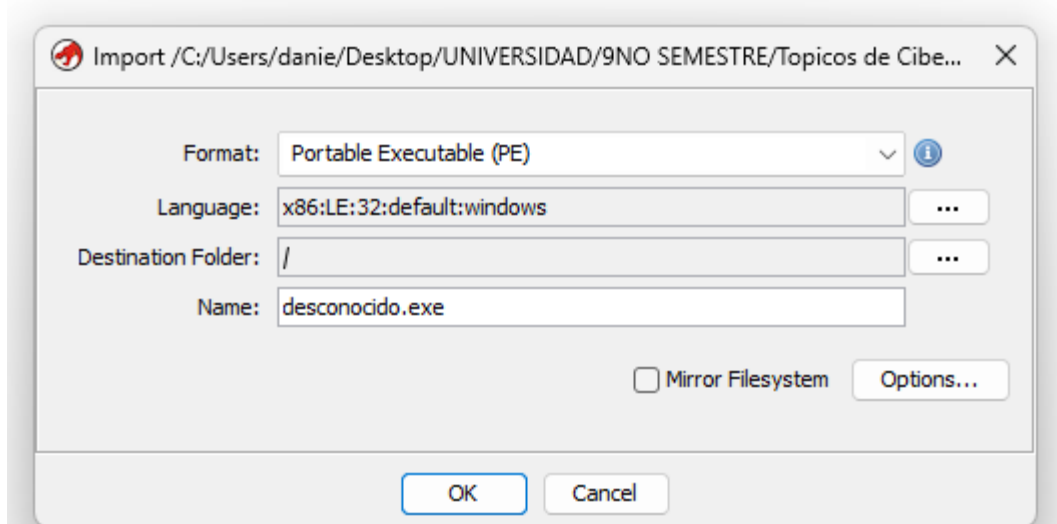


Figura 6. Detección automática del formato *Portable Executable (PE)* de 32 bits, arquitectura *x86*.

Al terminar la importación se mostró la ventana *Import Results Summary* con el resumen del archivo: tamaño de 2048 bytes, 14 funciones, hashes MD5 y SHA-256, entre otros datos.

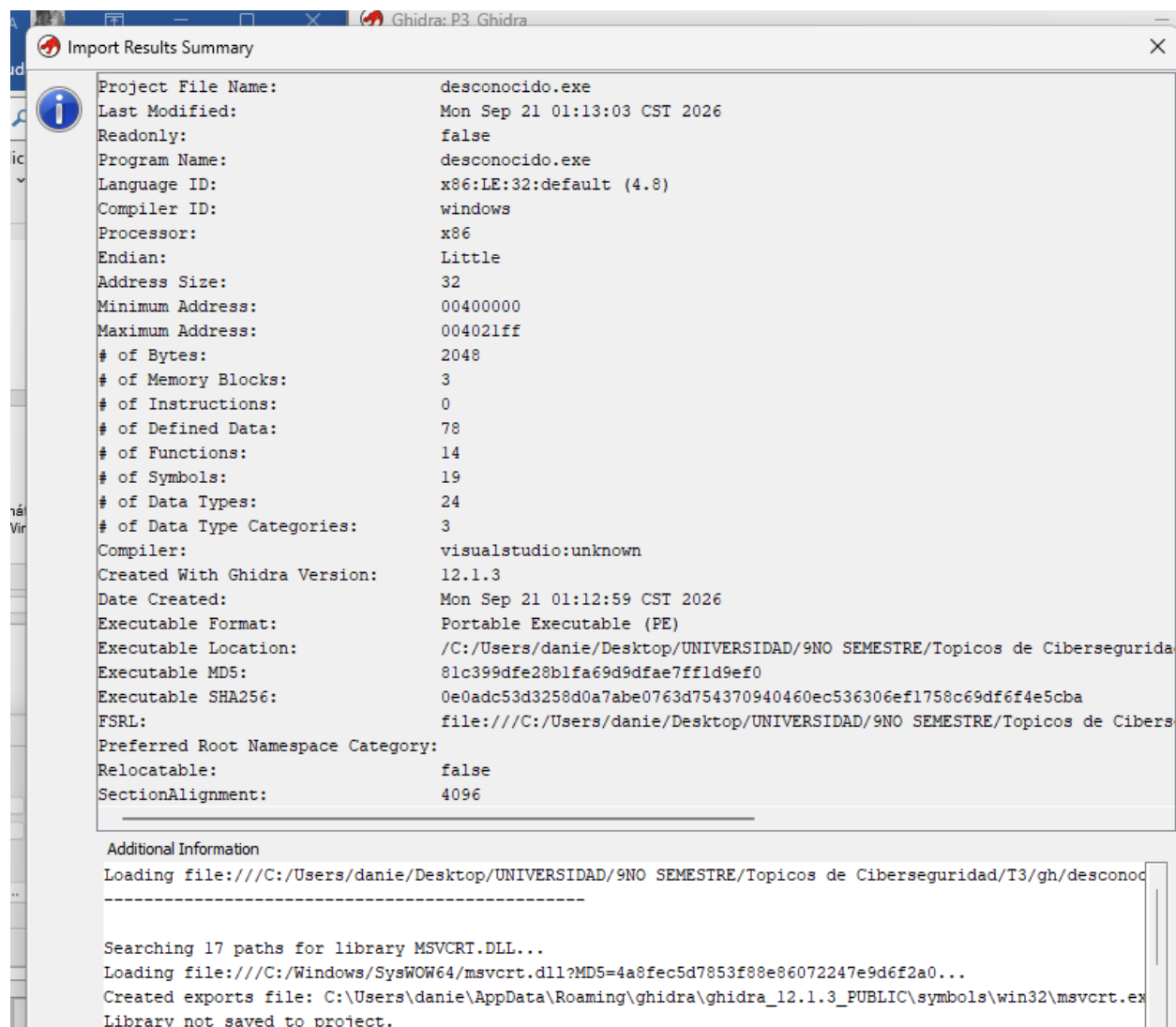


Figura 7. Ventana *Import Results Summary* con el resumen del archivo importado.

3.4 Análisis automático y localización del main

Se abrió el archivo con la herramienta CodeBrowser (el dragoncito verde del Tool Chest). Ghidra preguntó si se deseaba analizar el binario; se aceptó con Yes, dejando las opciones de análisis marcadas por defecto, y se ejecutó Analyze.

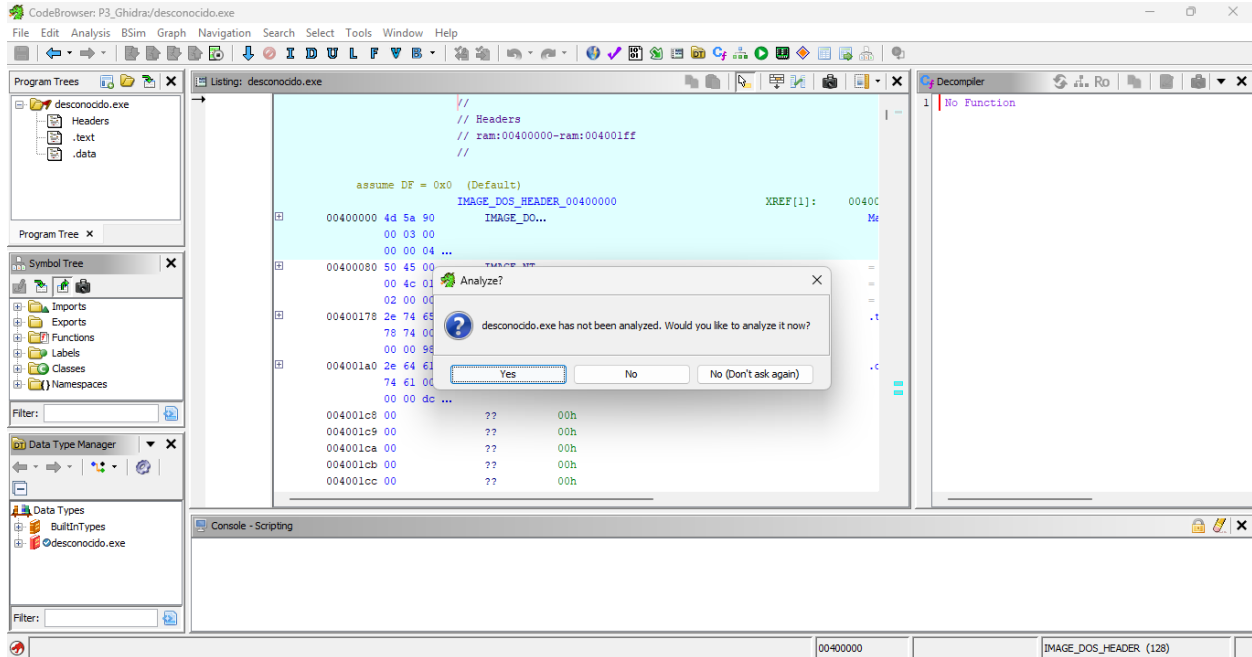


Figura 8. Cuadro de diálogo de análisis automático (Analyze) de Ghidra.

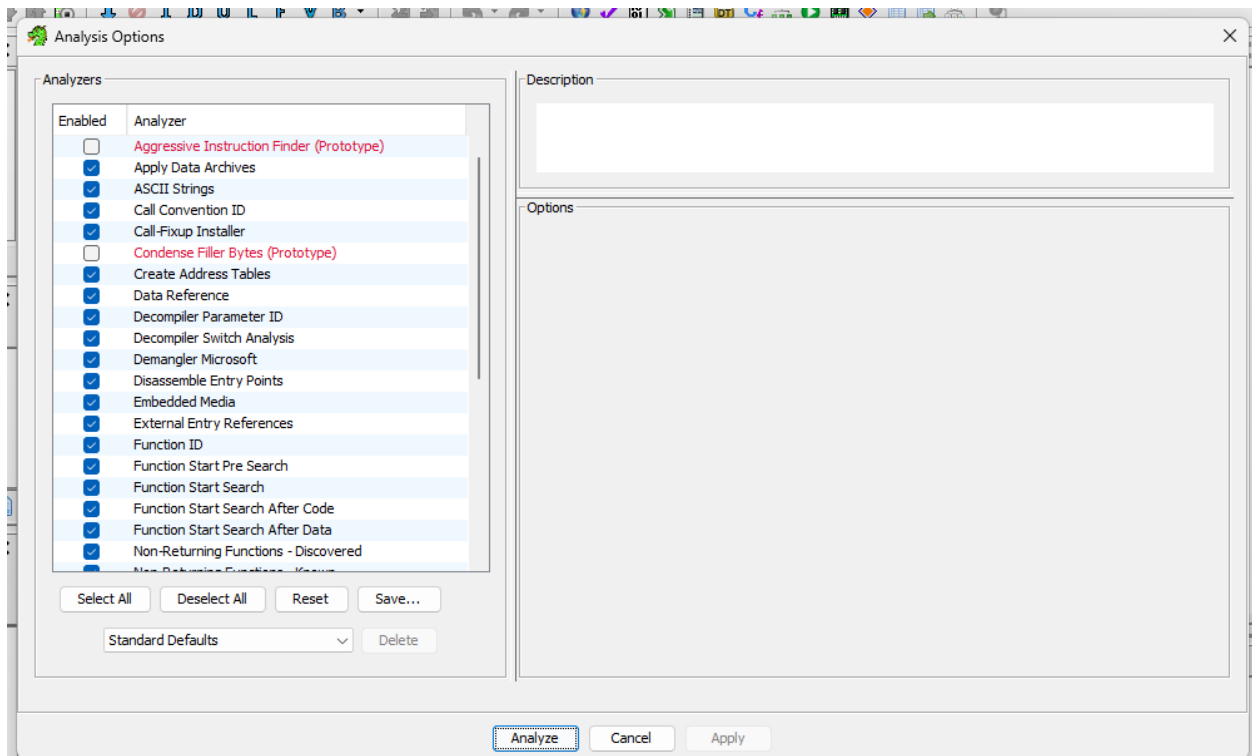


Figura 9. Opciones de análisis marcadas por defecto antes de ejecutar Analyze.

En Symbol Tree → Functions se identificó que la primera función, entry, corresponde al código de arranque que agrega automáticamente el compilador (TCC) para preparar el programa

antes de ejecutarlo; no es el main original. Para localizar la función principal real se buscó, dentro de entry, cuál función se invoca justo antes de la llamada a exit: en lenguaje C el código de arranque siempre recibe el valor que retorna main y se lo pasa a exit, por lo que esa función intermedia (FUN_00401000) es, con certeza, la función principal del programa.

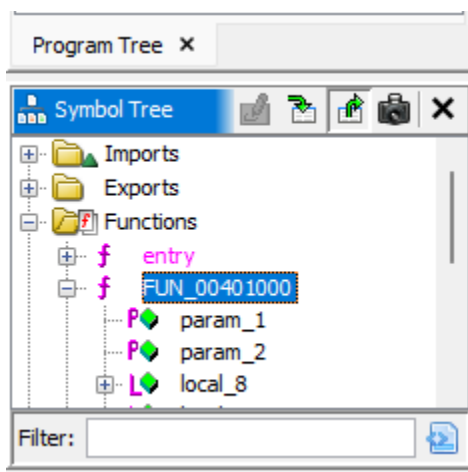


Figura 10. *Symbol Tree, carpeta Functions, mostrando FUN_00401000 seleccionada.*

3.5 Decompilación del pseudocódigo

Al abrir FUN_00401000 en el Decompiler, Ghidra mostró su pseudocódigo con nombres genéricos (param_1, param_2, local_8, undefined4, etc.), ya que no puede inferir por sí solo el propósito real de cada variable.

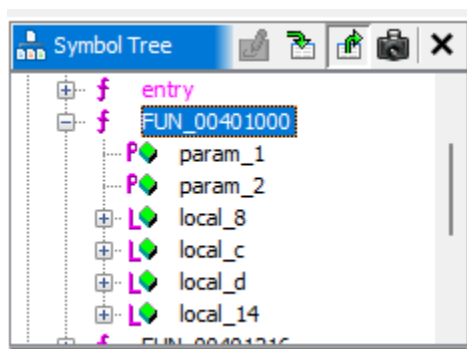


Figura 11. *Symbol Tree expandido: FUN_00401000 con sus parámetros y variables locales.*

Mediante Edit Function Signature se corrigió la firma de la función: se le asignó el nombre main, tipo de retorno int, y se renombraron los parámetros como argc (int) y argv (char **), que es la convención estándar en C para los argumentos que un programa recibe desde la línea de

comandos. Al aceptar el cambio, el pseudocódigo se volvió considerablemente más legible: Ghidra reconoció argv como un arreglo de cadenas y comenzó a representar el acceso a memoria como argv[1] en lugar de una operación aritmética sobre punteros.

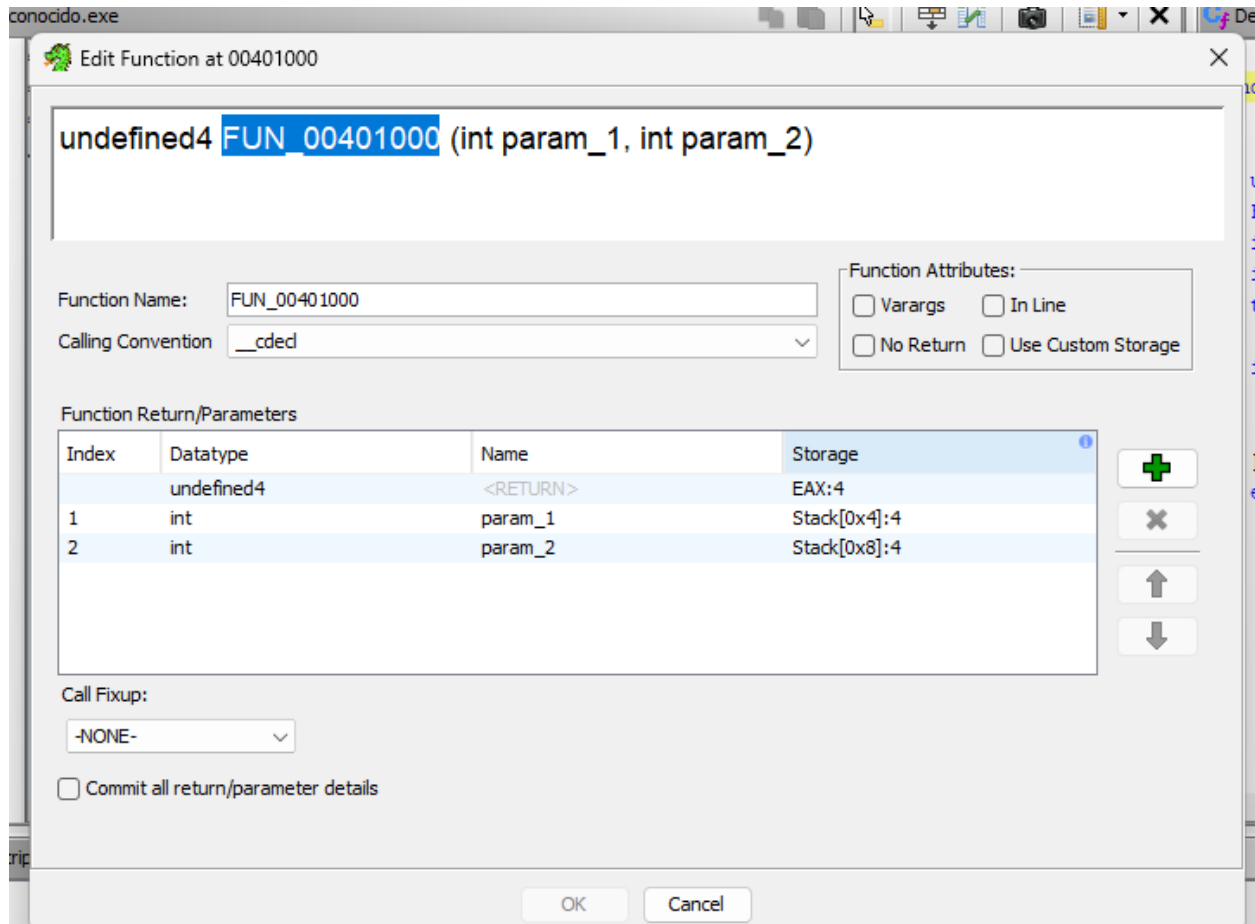


Figura 12. *Cuadro Edit Function Signature antes de la corrección.*

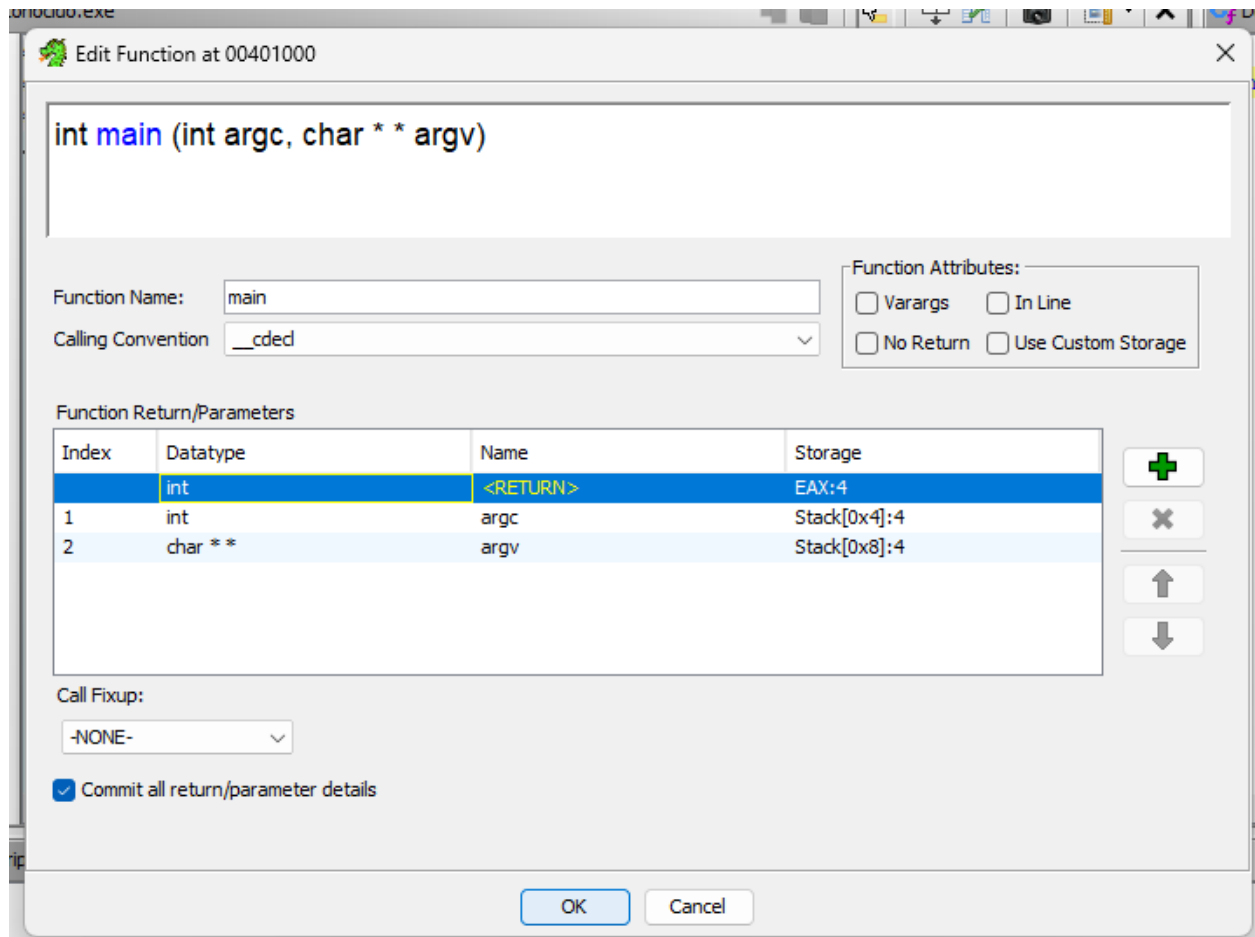


Figura 13. Cuadro Edit Function Signature después de la corrección (main, int argc, char **argv).

3.6 Escritura del código fuente en C

Con la lógica ya comprendida, se redactó manualmente el archivo prueba.c, traduciendo el pseudocódigo generado por Ghidra a código C estándar y compilable:

```
#include <stdio.h>
#include <stdlib.h>
#include <time.h>

int main(int argc, char *argv[])
{
    int iVar1;
    FILE *_File;
    int iVar2;
    time_t tVar3;

    if (argc < 2)
    {
        printf("Usage:\nsumnotransfer archivo");
        iVar1 = 1;
    }
    else
    {
        tVar3 = time((time_t *)0);
        srand((unsigned int)tVar3);

        _File = fopen(argv[1], "rb");
        if (_File == NULL)
        {
            printf("No se puede abrir el archivo");
            iVar1 = 1;
        }
        else
        {
            do
```

Figura 14. Archivo *sumnotransfer.c* abierto en el editor de código (parte 1).

```
{
    iVar1 = fgetc(_File);
    if (iVar1 != -1)
    {
        iVar2 = rand();
        iVar2 = iVar2 % 0x100;

        if (iVar1 < iVar2)
        {
            printf("%d-%d\n", iVar2, iVar2 - iVar1);
        }
        if (iVar2 < iVar1)
        {
            printf("%d+%d\n", iVar1 - iVar2, iVar2);
        }
        if (iVar1 == 0)
        {
            printf("1-1\n");
        }
        if (iVar2 == iVar1)
        {
            printf("%d+1\n", iVar1 - 1);
        }
    }
} while (iVar1 != -1);

fclose(_File);
iVar1 = 0;
}
return iVar1;
}
```

Figura 15. Archivo *sumnotransfer.c* abierto en el editor de código (parte 2).

4. Desarrollo de R004 - Compilación y comparación

4.1 Compilación del código reconstruido

El archivo *prueba.c* se compiló con TCC para obtener *reconstruido.exe*:

```
tcc.exe -o reconstruido.exe prueba.c
```

```
C:\Windows\System32\cmd.e X + v
(c) Microsoft Corporation. Todos los derechos reservados.

C:\tcc>tcc.exe -o reconstruido.exe prueba.c

C:\tcc>dir
El volumen de la unidad C no tiene etiqueta.
El número de serie del volumen es: 668D-D5C9

Directorio de C:\tcc

21/09/2026 01:55 a. m. <DIR> .
01/09/2026 07:39 p. m. <DIR> doc
01/09/2026 07:49 p. m. 2,048 ejemplo.exe
03/09/2026 08:04 p. m. 2,048 ejemplo2026.exe
03/09/2026 08:07 p. m. 2,048 ejemplo2026_parche.exe
01/09/2026 07:39 p. m. <DIR> examples
01/09/2026 07:44 p. m. 239 hackame0.c
03/09/2026 07:29 p. m. 2,048 hackame0.exe
03/09/2026 07:28 p. m. 2,048 hackame0.exe.bak
01/09/2026 07:39 p. m. <DIR> include
01/09/2026 07:39 p. m. <DIR> lib
01/09/2026 07:42 p. m. <DIR> libtcc
01/09/2026 07:39 p. m. 148,992 libtcc.dll
01/09/2026 08:38 p. m. 2,048 Parche.exe
01/09/2026 08:45 p. m. 2,048 Parche2.exe
03/09/2026 07:29 p. m. 2,048 Parche3.exe
03/09/2026 08:05 p. m. 555 parchep.py
21/09/2026 01:51 a. m. 1,483 prueba.c
01/09/2026 07:49 p. m. 2,048 prueba2026.exe
21/09/2026 01:55 a. m. 2,048 reconstruido.exe
```

Figura 16. *Compilación de prueba.c con TCC para generar reconstruido.exe.*

4.2 Comprobación funcional

Se creó un archivo de prueba, prueba.txt, y se ejecutaron tanto desconocido.exe como reconstruido.exe con el mismo archivo de entrada:

```
desconocido.exe PruebaL.txt
reconstruido.exe PruebaL.txt
```

```
C:\Users\danie\Desktop\UNIVERSIDAD\9NO SEMESTRE\Topicos de Ciberseguridad\T3\gh>desconocido.exe PruebaL.txt
107-27
78+36
69+48
147-46
31+67
8+89
149-117
157-88
191-78
27+90
115-10
150-38
25+86
79-47
172-64
238-141
136-36
45+69
10+95
26+82
74+34
5+106
```

Figura 17. Ejecución de *desconocido.exe* con el archivo *prueba.txt*.

```
C:\tcc>reconstruido.exe PruebaL.txt
241-161
145-31
54+63
119-18
45+53
36+61
17+15
203-134
67+46
180-63
195-90
164-52
107+4
78-46
226-118
188-91
154-54
67+47
147-42
241-133
6+102
201-90
```

Figura 18. Ejecución de *reconstruido.exe* con el archivo *prueba.txt*.

El programa utiliza números aleatorios (`srand(time(NULL))`), por lo que produce una salida distinta en cada ejecución, aunque procese el mismo archivo. Por esta razón no es posible

comparar el texto impreso de forma literal entre ambos programas; la comparación funcional válida es que ambos generen el mismo formato de salida (r-diferencia / diferencia+r), el mismo caso especial "1-1" cuando el byte leído es 0, y el mismo número de líneas procesadas para el mismo archivo de entrada.

4.3 Comparación de tamaño y hash SHA-256

Se compararon los tamaños de desconocido.exe y reconstruido.exe, y se calculó el hash SHA-256 de cada ejecutable con certutil:

```
dir desconocido.exe reconstruido.exe
certutil -hashfile desconocido.exe SHA256
certutil -hashfile reconstruido.exe SHA256
```



```
C:\Users\danie\Desktop\UNIVERSIDAD\9NO SEMESTRE\Topicos de Ciberseguridad\T3\gh>dir desconocido.exe reconstruido.exe
El volumen de la unidad C no tiene etiqueta.
El número de serie del volumen es: 668D-D5C9

Directorio de C:\Users\danie\Desktop\UNIVERSIDAD\9NO SEMESTRE\Topicos de Ciberseguridad\T3\gh
11/09/2026 06:10 p. m.                2,048 desconocido.exe

Directorio de C:\Users\danie\Desktop\UNIVERSIDAD\9NO SEMESTRE\Topicos de Ciberseguridad\T3\gh
21/09/2026 01:55 a. m.                2,048 reconstruido.exe
2 archivos                          4,096 bytes
0 dirs 266,673,872,896 bytes libres

C:\Users\danie\Desktop\UNIVERSIDAD\9NO SEMESTRE\Topicos de Ciberseguridad\T3\gh>certutil -hashfile desconocido.exe SHA256
SHA256 hash de desconocido.exe:
0e0adc53d3258d0a7abe0763d754370940460ec536306ef1758c69df6f4e5cba
CertUtil: -hashfile comando completado correctamente.

C:\Users\danie\Desktop\UNIVERSIDAD\9NO SEMESTRE\Topicos de Ciberseguridad\T3\gh>certutil -hashfile reconstruido.exe SHA256
SHA256 hash de reconstruido.exe:
ee7bb9e7d7284de1a1ed715de9650644a96ded4d3d7c863046bf48921b9c0262
CertUtil: -hashfile comando completado correctamente.
```

Figura 19. Resultado de dir y certutil -hashfile para ambos ejecutables.

Aunque ambos ejecutables pueden coincidir en tamaño (2048 bytes, al compilarse con el mismo TCC), los hashes SHA-256 son diferentes. Esto demuestra que los archivos no son idénticos byte a byte, aun cuando su comportamiento y lógica interna son equivalentes.

5. Resultados

- Ghidra reconoció correctamente desconocido.exe como un ejecutable PE de 32 bits para arquitectura x86.
- Se identificó FUN_00401000 como la función principal (main) mediante el rastreo de la llamada a exit desde entry.



- El Decompiler permitió reconocer la lógica completa: lectura de un archivo byte por byte, generación de un número aleatorio por cada byte, y cuatro posibles mensajes según la comparación entre ambos valores.
- El código reconstruido (prueba.c) fue compilado satisfactoriamente con TCC 0.9.27.
- desconocido.exe y reconstruido.exe presentan el mismo comportamiento funcional (mismo formato de salida y misma lógica), aunque no imprimen los mismos números por depender de la hora exacta de ejecución.
- Ambos ejecutables tienen el mismo tamaño, pero sus hashes SHA-256 son diferentes.

6. Conclusiones

La práctica permitió observar de forma directa cómo un ejecutable puede analizarse mediante ingeniería inversa para obtener una representación de alto nivel de su lógica. Ghidra facilitó la localización de la función principal —siguiendo la relación entre entry y exit— y la identificación de estructuras como el ciclo de lectura, las variables locales y las llamadas a funciones de la biblioteca estándar de C (fopen, fgetc, rand, printf, fclose).

La reconstrucción fue suficiente para crear un segundo programa con el mismo comportamiento observable: ambos ejecutables procesan un archivo byte por byte y generan el mismo tipo de salida. Sin embargo, la comparación demostró que equivalencia funcional no significa identidad binaria: aunque desconocido.exe y reconstruido.exe tienen el mismo tamaño, sus valores SHA-256.

Por lo tanto, el proceso de descompilación permite recuperar la lógica esencial de un programa, pero la compilación posterior puede producir instrucciones, organización interna y metadatos distintos a los del binario de referencia, incluso cuando se utiliza el mismo compilador.